



Universidad Nacional de La Matanza

Algoritmos y estructuras de datos

Trabajo práctico:

TaCti

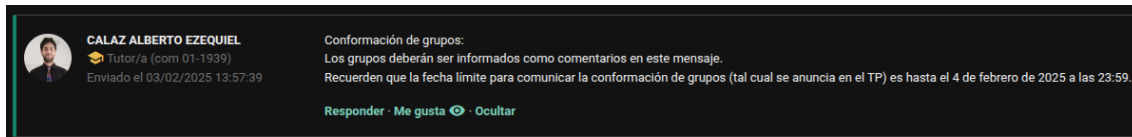


Modalidad de entrega

La entrega es grupal, en grupos de 4 personas.

En caso de conformarse todos los grupos necesarios y que aún haya estudiantes sin grupo, estos serán asignados por los docentes a grupos previamente armados, formando así grupos de 5 personas.

Los grupos deben ser formados y notificados vía foro de MIEL hasta las 23:59 del martes 4 de febrero de 2025 como comentarios en la siguiente publicación:



Respetar la fecha de conformación del grupo es condición para aprobar el TP.

El nombre de cada grupo debe ser una palabra, y no se puede repetir con otro grupo. El nombre del grupo debe ser una palabra que figure en el diccionario de la RAE, que pueden consultar en el siguiente link: <https://dle.rae.es/>

El nombre del grupo debe estar formado por letras cuyo valor ASCII esté comprendido entre 0x41 y 0x5a (inclusive). Eso implica que no se admiten espacios, letras en minúscula, tildes, números, etc.

Ejemplos de nombres que serán rechazados:

Palabra	Motivo de rechazo
LOS PIOJOS	Tiene un espacio (0x20)
La Renga	Tiene un espacio y letras en minúsculas
ASDF	Aviso: La palabra asdf no está en el Diccionario
C++	El ASCII de '+' es 0x2b, no está en el rango solicitado

Ejemplo de nombre válido:

Palabra	Motivo de aceptación
INVISIBLE	Está formado por caracteres válidos, y según la RAE su definición es: 1. adj. Que no puede ser visto.

Se deberá entregar un archivo con el siguiente formato:

TP_ALGORITMOS_2024_3C_{NOMBRE_DE_GRUPO}.zip

Por ejemplo: Si el grupo se llamase "INVISIBLE" -y sus integrantes fueran Spinetta, Pomo, Machi y Gubitsch- el archivo debería llamarse
TP_ALGORITMOS_2024_3C_INVISIBLE.zip

Se deberán eliminar las carpetas bin y obj.



En la entrega grupal por míel, se deberá adjuntar el .zip + la URL al repositorio GitHub.

El repositorio debe ser público.

El formato de entrega es motivo de rechazo del TP.

Como ocurre con cualquier sistema, debe respetar el formato solicitado.

Ejemplos de archivos que se considerarán incorrectos:

Nombre de archivo	Motivo de rechazo
TP_ALGORITMOS_2024_3c_INVISIBLE.zip	contiene una c minúscula en el nombre
TP_ALGORITMOS_2024_3C_INVISIBLE(1).zip	contiene (1) en su nombre
TP_ALGORITMOS_2024_2C_INVISIBLE.zip	cuatrimestre incorrecto
TP_ALGORITMOS_2024_3C_INVISIBLE.rar	formato de archivo incorrecto

En el cronograma de clases figuran las fechas de entrega y defensa del TP. La defensa del TP, es una instancia de evaluación de la materia.

Necesidad

Un grupo de desarrolladores está creando un tótem interactivo para entretenimiento en supermercados. Como parte del proyecto, quieren incluir un minijuego de **Tateti (Ta-C-ti)** en el que los usuarios puedan jugar contra una inteligencia artificial básica. Cada partida se registrará en un servidor remoto mediante una API para analizar los resultados y mejorar la IA en futuras versiones.

Se requiere implementar el juego en **C**, permitiendo a los usuarios jugar partidas individuales contra la máquina, registrar los resultados en una API y generar un informe local con estadísticas.

Reglas del juego

Al inicio de cada juego, se va a:

- Ingresar los nombres de los jugadores
- Sortear aleatoriamente el orden de los jugadores

Las reglas del juego son las siguientes:

- **Turnos alternados:** El usuario y la máquina juegan por turnos en un tablero de **3x3**.
- **Condiciones de victoria:**
 - Un jugador gana si coloca tres de sus símbolos en una línea horizontal, vertical o diagonal.
 - Si el tablero se llena sin que haya un ganador, la partida se considera un empate.



- **Estrategia de la máquina:** La máquina jugará con una estrategia predefinida, como:
 - Elegir aleatoriamente si no hay una jugada clara.
 - Bloquear la victoria del jugador si es posible.
 - Ganar en la siguiente jugada si tiene la oportunidad.

Consigna

Implementar el juego **Tateti en C** con las siguientes características:

Apenas se inicie el programa, deberá haber un menú de 3 opciones:

[A] Jugar

[B] Ver ranking equipo

[C] Salir

Si alguien ingresa a “Jugar”, primero se le pedirá que cargue los nombres de las personas que van a jugar. Puede ingresar la cantidad de nombres que desee.

Una vez que termine de ingresarlos, aparecerá por pantalla el orden en el que jugarán los jugadores (recordar que se determina aleatoriamente), y se le preguntará al primer jugador si está listo. En caso de que sí, inicia el juego.

Cada jugador va a jugar X cantidad de partidas (determinada por el archivo de configuración). En cada partida, le tocará aleatoriamente ser la X o el O. Se mostrará el tablero y el jugador tiene que ingresar donde quiere hacer su movimiento. La máquina responde con su jugada. Se repite el proceso hasta que uno de los dos gane o se empate.

Cuando el jugador termine con sus partidas, pasará al siguiente jugador. Y así sucesivamente hasta terminar todas las partidas de todos los jugadores. Por cada partida que se termine, se sumaran +3 puntos si gana el jugador, +2 puntos si empata con la maquina y -1 punto si pierde

Una vez finalizadas las partidas, se generará un informe con el detalle de las partidas (como quedó el tablero al final), quien ganó, el puntaje de cada una, el puntaje total por jugador y el resultado final, qué jugador/es obtuvieron mayor puntaje. . El nombre del archivo debe contener la fecha y la hora actual en el siguiente formato: YYYY-MM-DD-HH-mm. Ejemplo de nombre: informe-juego_2024-02-01-12-20.txt. Además, se enviará a una API el resultado de los jugadores con el siguiente formato:

```
{
  "codigoGrupo": "ASD123",
  "jugadores": [
    {
      "nombre": "Juan",
      "puntos": 10,
    }
  ]
}
```



Las configuraciones de la api + la cantidad de partidas por jugador se leerá de un archivo txt con el siguiente formato:

UrlApi | Codigo identificador grupo

Cantidad de partidas

<https://api.com> | ASD123

3

Por otro lado, deberán subir el código a un repositorio en GitHub. Deberán agregar a su repo un readme que indique cómo jugar el juego y qué hacer si quiero cambiar las configuraciones del juego.

A ese repo se deberá subir un documento con diferentes lotes de prueba con el siguiente formato:

Descripción	Salida esperada	Salida obtenida
Se quiere probar qué es lo que pasaría si....	Se espera que....	La salida obtenida fue....

Mínimo se deben documentar 8 casos de prueba, con captura de pantalla de la salida obtenida.

Condiciones básicas para aprobar

- ✓ Código prolijo, dividido en funciones
- ✓ Funciones lo más genéricas posibles
- ✓ Nombres significativos de variables
- ✓ 0 errores 0 warnings
- ✓ Que funcione mínimo para todos los casos de prueba que presentan



Detalle de Endpoints

Esta API permite gestionar rankings de jugadores. Los endpoints disponibles son: **GET**, **POST** y **DELETE**. A continuación, se detallan sus funciones, ejemplos y respuestas.

1. Obtener Rankings (GET)

Descripción:

Obtiene los rankings de jugadores para el código de grupo enviado. Si no hay datos en el grupo, devuelve un array vacío.

Endpoint:

GET <https://algoritmos-api.azurewebsites.net/api/TaCTi/{CodigoGrupo}>

Respuesta:

- **200 OK:** Devuelve un array con los rankings del grupo.
- **400 Bad Request:** Si el grupo no existe o ocurre un error.

Ejemplo de respuesta:

```
[
  {
    "nombreJugador": "giselle",
    "puntaje": 10,
    "ultimaPartidaJugada": "04/02/2025 21:37:33"
  },
  {
    "nombreJugador": "matias",
    "puntaje": 2,
    "ultimaPartidaJugada": "04/02/2025 21:37:33"
  }
]
```

Si el grupo está vacío:



1

Ejemplo algoritmos / **Obtener ranking**

GET <https://algoritmos-apl.azurewebsites.net/api/TaCTI/prueba> Send

Params Authorization Headers (8) Body Scripts Tests Settings Cookies

Query Params

Key	Value	Description	Bulk Edit
-----	-------	-------------	-----------

Body Cookies (2) Headers (7) Test Results 200 OK · 291 ms · 441 B Save Response

JSON Preview Visualize

```
1 [
2   {
3     "nombreJugador": "giselle",
4     "puntaje": 10,
5     "ultimaPartidaJugada": "04/02/2025 21:37:33"
6   },
7   {
8     "nombreJugador": "matias",
9     "puntaje": 2,
10    "ultimaPartidaJugada": "04/02/2025 21:37:33"
11  }
12 ]
```

GET <https://algoritmos-apl.azurewebsites.net/api/TaCTI/prueba> Send

Params Authorization Headers (8) Body Scripts Tests Settings Cookies

Query Params

Key	Value	Description	Bulk Edit
-----	-------	-------------	-----------

Body Cookies (2) Headers (7) Test Results 200 OK · 259 ms · 351 B Save Response

JSON Preview Visualize

```
1 []
```

Ejemplo algoritmos / **Obtener ranking**

GET <https://algoritmos-apl.azurewebsites.net/api/TaCTI/prueba11> Send

Params Authorization Headers (8) Body Scripts Tests Settings Cookies

Query Params

Key	Value	Description	Bulk Edit
-----	-------	-------------	-----------

Body Cookies (2) Headers (5) Test Results 400 Bad Request · 218 ms · 204 B Save Response

Raw Preview Visualize

```
1 Grupo no encontrado
```



2. Guardar Rankings (POST)

Descripción:

Guarda jugadores y sus puntajes en el grupo especificado. Si el nombre del jugador ya existe en el grupo, acumula el puntaje.

Endpoint:

POST <https://algoritmos-api.azurewebsites.net/api/TaCTi>

Cuerpo de la solicitud:

```
{
  "CodigoGrupo": "PRUEBA",
  "Jugadores": [
    {
      "nombre": "giselle",
      "puntos": 10
    },
    {
      "nombre": "matias",
      "puntos": 2
    }
  ]
}
```

Respuesta:

- **204 No Content:** Los datos fueron guardados correctamente.
- **400 Bad Request:** Si el grupo no existe o hay algún problema con los datos enviados.



Ejemplo algoritmos / Guardar ranking

POST <https://algoritmos-apl.azurewebsites.net/api/TaCTI> Send

Params Authorization Headers (10) Body Scripts Tests Settings Cookies Beautify

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL ☐ JSON

```
1 {
2   "CodigoGrupo": "prueba",
3   "Jugadores": [
4     {
5       "nombre": "giselle",
6       "puntos": 10
7     },
8     {
9       "nombre": "matias",
10      "puntos": 2
11     }
12   ]
13 }
```

Body Cookies (2) Headers (3) Test Results 204 No Content · 430 ms · 115 B Save Response

Raw Preview Visualize

1

Ejemplo algoritmos / Guardar ranking

POST <https://algoritmos-apl.azurewebsites.net/api/TaCTI> Send

Params Authorization Headers (10) Body Scripts Tests Settings Cookies Beautify

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL ☐ JSON

```
1 {
2   "CodigoGrupo": "prueb22a",
3   "Jugadores": [
4     {
5       "nombre": "giselle",
6       "puntos": 10
7     },
8     {
9       "nombre": "matias",
10      "puntos": 2
11     }
12   ]
13 }
```

Body Cookies (2) Headers (5) Test Results 400 Bad Request · 214 ms · 204 B Save Response

Raw Preview Visualize

1 Grupo no encontrado

Ejemplo algoritmos / Guardar ranking

POST <https://algoritmos-apl.azurewebsites.net/api/TaCTI> Send

Params Authorization Headers (10) Body Scripts Tests Settings Cookies Beautify

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL ☐ JSON

```
1 {
2   "CodigoGrupo": "prueba",
3   "Jugadores": [
4     {
5       "nombre": "giselle;;",
6       "puntos": 10
7     },
8     {
9       "nombre": "matias",
10      "puntos": 2
11     }
12   ]
13 }
```

Body Cookies (2) Headers (5) Test Results 400 Bad Request · 232 ms · 228 B Save Response

Raw Preview Visualize

1 NombreJugador tiene caracteres no aceptados



3. Resetear Grupo (DELETE)

Descripción:

Elimina todos los rankings de un grupo específico, reseteándolo completamente. Este método NO debe estar implementado en su código, es solamente para que puedan limpiar los datos que hayan cargado.

Endpoint:

DELETE <https://algoritmos-api.azurewebsites.net/api/TaCTi/{CodigoGrupo}>

Respuesta:

- **200 OK:** El grupo fue reseteado correctamente.
- **400 Bad Request:** Si el grupo no existe o ocurre un error.

